

AI Workflow Audit — Accounts Payable (Three-Way Match)

A reproducible proof-of-concept for **auditing an AI automation opportunity before building it**: decompose a workflow, score each step on impact × risk, measure the risky steps empirically in a sandbox, and quantify the dollar value of the right operating model (usually hybrid, not full automation).

POC #1 of a three-part series: manufacturing AP → finance/banking fraud → healthcare (HIPAA).

Why this exists

Most enterprise AI pilots fail on integration and risk, not model quality. The fix is a diagnostic *before* the build: which steps to automate, which to run hybrid (AI proposes, human approves), and which to leave manual — scored on business impact and the cost of an AI error. This repo demonstrates that diagnostic end to end on a manufacturing accounts-payable workflow.

What it shows

On a synthetic three-way-match dataset, the candidate automation matches invoices at **99.4% accuracy** but catches only **50% of fraud**. Because a paid fraudulent invoice is irreversible, the value model shows **naive full automation nets ~\$0.10M/yr while a hybrid model with a human payment gate nets ~\$3.28M/yr** (illustrative inputs). The audit's job is to find that gap before the client spends a quarter building the wrong thing.

Repository structure

```
ap-audit-poc/
├── README.md
├── data/
│   ├── generate_data.py      # reproducible synthetic dataset (seed=42)
│   ├── vendors.csv           # vendor master incl. bank details
│   ├── purchase_orders.csv   # what we agreed to buy
│   ├── goods_receipts.csv    # what was received
│   ├── invoices.csv          # what the vendor billed
│   └── ground_truth.csv      # labeled disposition per invoice (the tail)
├── harness/
│   ├── match_engine.py       # automation under test (rules; LLM/MCP in prod)
│   ├── run_audit.py          # offline-eval: scores engine vs ground truth
│   └── results.json           # emitted metrics (feeds the value model)
└── reports/
    ├── audit_report.md        # the consulting deliverable
    ├── build_value_model.py   # builds the Excel model from results.json
    └── value_model.xlsx       # impact×risk scoring + value equation (formulas)
```

How to run

```
pip install pandas openpyxl
cd data      && python generate_data.py      # writes the CSVs
cd ../harness && python run_audit.py          # prints metrics, writes results.json
cd ../reports && python build_value_model.py  # writes value_model.xlsx
```

The edge-case tail (what makes this a real audit)

The dataset deliberately includes the exceptions that break naive automation: price mismatch, quantity mismatch, duplicate invoice, missing PO, tax error, and **fraud (vendor bank-detail change / payment redirect)**. The harness reports per-category recall so the risk score is measured, not guessed.

Production note (MCP)

The audit itself is tool-agnostic. The *implementation* would give the agent permissioned access to ERP, vendor master, and banking systems through MCP tools. Every write-capable tool (`release_payment` , `update_vendor_bank`) is both an attack surface and the high-risk cell of the matrix — so the MCP tool scope is audited alongside the workflow.

License / use

Portfolio demonstration. Synthetic data only; no real client information.